IUCAA CRAY CX1 supercomputer

Users Guide (v1.0)

Jayanti Prasad
Inter-University Centre for Astronomy & Astrophyscs (IUCAA) India

November 17, 2011

1 Introduction

Cray CX1 is a desktop/deskside supercomputer with 1 head node, 1 GPU node and 4 compute nodes. Each node has two 2.67 Ghz Intel Xeon(5650) processors, with every processor having six cores. In total there are 72 cores on the systems. Since there are twelve cores on every node, one can use twelve OpenMP hardware threads. Every node has 24 GB RAM and in total the system has 144 GB RAM. At present there is around 5 TB disk space available for users on three different partitions, named /home, /data1/ and /data2. Keeping in mind that this is a small system and not more than ten users will be using it, every user can get 100 GB space in the home area. Data which is used by many users like WMAP data can be kept in a common area.

Names of the nodes are as **hpccmb** (head node), **compute-00-00** (GPU node) and **compute-00-01, compute-00-02** and , **compute-00-04**.

There are many software, packages and libraries already installed in the system and more (open source) can be installed. At present the system has the following compilers:

1. gcc version 4.1.2 (**gfortran** and **gcc**).
2. Compilers supplied with the *platform* system (**mpicc**, **mpiCC**, **mpif77**, **mpif90** etc.,) are installed in "/opt/platform_mpi/".
3. Open MPI compilers (**mpic++ ,mpicc, mpiCC, mpicxx, mpiexec, mpif77** etc.,) installed in " /data1/software/openmpi".
4. MPICH 2 is installed in "/data1/software/mpich2".
5. Intel compilers (**icc** and **ifort**) are installed in "/opt/intel/". In this area Intel Mathematical Kernel Library (MKL) and Intel Threading Building Blocks (ITBB) are also installed.

Platform LSF is successfully running on the system and by default all MPI jobs will be assigned in a "batch queue" mode. In general, users are not needed to specify the nodes to which they want to assign the job, however, it may be useful in some cases.

The system already has many scientific packages/libraries pre-installed or have been installed some of them are as follows:

1. **blacs, linpack, scalapack** and **hdf5** supplied with the system are installed in "/opt".
2. **gnuplot, gv, acoread** etc., are installed.
3. A good number of packages including **fftw2, fftw3, pgplot, cfitsio, lapack** are installed in "/data1/software".
4. Some of the packages related to CMBR work (**cmbfast, camb, cosmomc, healpix etc.,**) are already been installed in "/data1/soft_cmbr/" and more will be installed.

2 Job Submission

Job on the system can be submitted using two different modes. In the first case job can be submitted using a *platform* GUI supplied with the system which can be accessed at the following URL from the IUCAA network.

`http://192.168.11.243:8080/platform/,`

it needs a login and password.

Jobs can be submitted from the terminal also and a typical MPI submission script may look like:

```
mpirun  -np 32 -hostlist "compute-00-00,compute-00-01,compute-00-02,compute-00-04"  \
./cosmomc params.ini
```

apart from this the direct submission also works:

```
mpirun -np 4 ./a.out
```

I think one of the most useful ways to submit a MPI job, particularly when it uses OpenMP also is by using the *appfile*. Create an appfile with the following content:

```
cat appfile
-h compute-00-01 -np 2 ./cosmomc params.ini
-h compute-00-02 -np 2 ./cosmomc params.ini
-h compute-00-03 -np 2 ./cosmomc params.ini
-h compute-00-04 -np 2 ./cosmomc params.ini
```

Once appfile is created it can be run using the following command.

```
mpirun -f appfile
```

This is the content of an appfile which will be used to run "cosmomc" with eight chains, two on every node. Note that the use of appfile makes it quite straightforward to use MPI and OpenMP together. For example, in the case of cosmomc, if one runs only two MPI process on a node then we can let every process to use six OpenMP threads when computing C_ls using "camb".

Once a job is submitted it can be monitored using the usual command **bjobs** and can be killed using the command **bkill**. It is recommended that after the submission user login into the machines on which the jobs are running and check the status using **top** command.

Note that MPI jobs do not print any output on the standard output and one needs to wait the files to be written. However, the standard output can be seen using the command **bpeek** at any stage.

3 GPU Computing

The system has one Nvidia Tesla C2050 GPU which is hosted on the node **compute-00-00**. Detail status of the GPU can be printed on the standard output using the the **nvidia-smi** command one of its example is as follows:

```
nvidia-smi  -q
```

This will print a lot of useful information about the GPU. You can use "–help" option also to find all the option available for **nvidia-smi**.

Compute Unified Device Architecture (CUDA) library as well as the compiler **nvcc** is installed in the are "/usr/local/cuda". In order to run a cuda program, follow the following three steps:

- Write a source/program file with suffix ".cu" following the cuda syntax.
- Compile the program with **nvcc**

  ```
  nvcc prog.cu
  ```

- Run the executable

  ```
  ./a.out
  ```

I have written a program called 'mygpu.cu' if run, gives the following information about the GPU.

```
nvcc mygpu.cc
./a.out
 --- General Information for device 0 ---
Name: Tesla C2050
Compute capability: 2.0
Clock rate: 1147000
Device copy overlap: Enabled
Kernel execition timeout : Disabled
 --- Memory Information for device 0 ---
Total global mem: 2817982464
Total constant Mem: 65536
Max mem pitch: 2147483647
Texture Alignment: 512
 --- MP Information for device 0 ---
Multiprocessor count: 14
Shared mem per mp: 49152
Registers per mp: 32768
Threads in warp: 32
Max threads per block: 1024
Max thread dimensions: (1024, 1024, 64)
Max grid dimensions: (65535, 65535, 65535)
```

Useful References

http://www.platform.com/workload-management/high-performance-computing
http://www-cdf.fnal.gov/offline/runii/fcdfsgi2/lsf_batch.html
http://www.iucaa.ernet.in/ jayanti/parallel.html